Since most unit primitives map 1:1 to an interface element (like a button or API call), many service definitions only define a single unit primitive of the same name.
  - This creates redundancy and noise in the configuration files.
  - Example: summarizecomments

Configuration management challenges

Updating unit primitive configurations currently requires manual changes in multiple places:

- GitLab rails: Update the config/accessdata.yml configuration and permission checks.
- CustomersDot: Update the config/cloudconnector.yml configuration.
- AI gateway and Duo workflows: Modify unit primitive mappings and access control logic.

Current problems

- No Single Source of Truth:
  - Configurations are scattered across several repositories and services, leading to inconsistencies and making management difficult. Multiple merge requests across different repositories add complexity and friction.

Decision

We will implement a new unit primitive-based configuration system by extracting it to an external library (gitlab-cloud-connector) that will serve as the Single Source of Truth (SSoT). This library will be available as both a Ruby gem and a Python package.

Configuration structure

shell
config
·· unitprimitives/
· ·· duochat.yml
· ·· ...
·· backendservices/
· ·· aigateway.yml
· ·· ...
·· addons/
· ·· duopro.yml
· ·· ...
·· licensetypes/
   ·· premium.yml
   ·· ...

Unit primitives

We have a .yml file per unit primitive. It contains information on how this unitprimitive is bundled with add-ons and license types, and other useful metadata.

yaml

config/unitprimitives/duochat.yml

name: aigateway
projecturl: https://gitlab.com/gitlab-org/modelops/applied-ml/code-suggestions/ai-assist
group: group::ai framework
jwtaud: gitlab-ai-gateway

Add-ons and license types

Similarly defined in their respective YAML files, providing necessary metadata.

Expected process for adding/modifying unit primitives

Making changes

- Single update point: Developers modify unit primitive definitions by editing the YAML files in the gitlab-cloud-connector repository.
  - CLI tools: Command-line tools for adding, updating, and removing unit primitives should be implemented to simplify this step and make it less error-prone.
  - Example: To add a new unit primitive codereview, create config/unitprimitives/codereview.yml with the necessary configuration.

Deployment process

- Automated release pipeline:
  - Validation: Changes are validated using JSON schema validation in CI/CD pipelines.
  - Testing: Automated tests ensure consistency and correctness.
  - Publishing: New versions of the Ruby gem and Python package are published to RubyGems and PyPI upon successful validation and testing.

- Versioning: The library follows semantic versioning. Downstream applications are updated to use the new version.

Updating downstream applications

- GitLab Rails, CustomersDot, and others:
  1. Dependency update: Bump the gitlab-cloud-connector gem version in the Gemfile.
  2. Merge request: Submit an MR to update the dependency and any necessary code changes.
  3. Deployment: Changes are deployed following the standard GitLab deployment process.

How UI permissions will work

- Centralized permission checks:
  - Clients can check access to the most basic unit primitive to determine UI element availability. With the knowledge of the basic unit primitive, client still has the flexibility to check if the user has access to the UI element, without needing to understand all underlying unit primitives or configurations.
  - The actual permission logic that considers license types, add-on purchases and user seat assignments can be implemented in separate layer, based on fetched unit primitive

configuration.

Backward compatibility

- Server-Side mapping: Implement mapping from unit primitives to legacy services to maintain compatibility.
- Support for deprecated unit primitives: Continue to support deprecated unit primitives with appropriate deprecation notices.
- GraphQL schema updates:
  - Update the GraphQL CloudConnectorAccessType to include both availableServices and availableUnitPrimitives, deprecating availableServices but keeping it available until all clients have migrated.
  - Mark old unit primitives as deprecated in their YAML files instead of removing them, providing deprecation messages and timelines.

API for reading configuration

- Language-Agnostic library: The gitlab-cloud-connector library is available in both Ruby and Python.
- Backward compatibility: Provides an adapter to convert the new YAML format to the legacy services format for older systems.
- Permission agnostic: The library focuses on providing configuration information, leaving permission checks to the application logic.

Tools and validation

- CLI tools: Provide command-line tools for adding, updating, and removing unit primitives.
- Schema validation: Use JSON schema validation in CI/CD pipelines to ensure configuration correctness.
- Automated testing: Implement comprehensive test suites to validate changes.
- Automated release process: Streamline the release and publishing process to minimize delays.

Consequences

Positive

- Single source of truth: Centralized configuration simplifies maintenance and ensures consistent access across all systems.
- Modular and scalable: Modular YAML files allow easy addition of new unit primitives by stage groups, reducing the maintenance surface through a "fix it once, run everywhere" approach.
- Separation of concerns: Properly divides UI functionality and backend access control, keeping concerns distinct and manageable.
- Centralized ownership and configuration: Configuration management and ownership are centralized, making updates efficient and reducing potential discrepancies.

Negative

- Release dependency: Changes require new library releases, creating dependencies on release cycles.
  - Mitigation: Establish a streamlined release process and use automation to minimize delay.

- Reduced code visibility: Code is less visible to the broader team, potentially reducing collaborative input.
  - Mitigation: Enhance documentation and use internal communication channels to keep teams informed of changes.
- Security update challenges: Managing security updates may become more complex across multiple repositories.
  - Mitigation: Implement regular automated security scans and synchronize updates across repositories.
- Initial setup overhead: Setting up and maintaining a separate repository introduces initial overhead.
  - Mitigation: Invest in comprehensive setup documentation and tooling to simplify onboarding and maintenance.

Next steps

Core library development

- Create gitlab-cloud-connector library with the new YAML configuration structure.
- Implement configuration parser and validator.
- Build adapter to convert new YAML format to legacy services format.
- Add basic test suite and CI/CD pipeline.
- Create configuration management CLI for adding, updating, and removing unit primitives.

Migration path

- Update all GitLab Rails and CustomersDot instances to use the new library (SSoT).
- Switch primary source to the new YAML format while maintaining legacy output.
- Gradually migrate clients to use the new YAML format and API, deprecating the legacy AvailableServices.
- Maintain backward compatibility with existing services format for older GitLab versions.
- Ensure continuity of impacted analytics data models by replacing services (renamed to features in the data platform and Tableau dashboards) with more granular attributes such as featurecategory (as detailed in issue 502457).

Documentation

- Create guides for unit primitive management.

Client updates

- Update permission checking mechanisms.
- Support both old and new formats for backward compatibility.

Useful links

- Decide on CloudConnector unit primitives config and API
- Extract CloudConnector unitprimitive configuration and logic

This decision maintains a two-way door, allowing for future changes if needed. We will focus on implementing this solution and integrate additional features and backends before considering

further improvements.

SECTION: engineering/architecture/design-documents/cloud_native_build_logs/_index.md

{{< engineering/design-document-header >}}

Cloud native and the adoption of Kubernetes has been recognised by GitLab to be one of the top two biggest tailwinds that are helping us grow faster as a company behind the project.

This effort is described in a more details in the infrastructure team handbook.

Traditional build logs

Traditional job logs depend a lot on availability of a local shared storage.

Every time a GitLab Runner sends a new partial build output, we write this output to a file on a disk. This is simple, but this mechanism depends on shared local storage - the same file needs to be available on every GitLab web node machine, because GitLab Runner might connect to a different one every time it performs an API request. Sidekiq also needs access to the file because when a job is complete, the trace file contents are sent to the object store.

New architecture

New architecture writes data to Redis instead of writing build logs into a file.

To make this performant and resilient enough, we implemented a chunked I/O mechanism - we store data in Redis in chunks, and migrate them to an object store once we reach a desired chunk size.

Simplified sequence diagram is available below.

```
mermaid
sequenceDiagram
    autonumber
    participant U as User
    participant R as Runner
    participant G as GitLab (rails)
    participant I as Redis
    participant D as Database
    participant O as Object store

    loop incremental trace update sent by a runner
      Note right of R: Runner appends a build trace
      R->>+G: PATCH trace [build.id, offset, data]
```

```
  G->>+D: find or create chunk [chunk.index]
  D-->>-G: chunk [id, index]
  G->>I: append chunk data [chunk.index, data]
  G-->>-R: 200 OK
end

Note right of R: User retrieves a trace
U->>+G: GET build trace
loop every trace chunk
  G->>+D: find chunk [index]
  D-->>-G: chunk [id]
  G->>+I: read chunk data [chunk.index]
  I-->>-G: chunk data [data, size]
end
G-->>-U: build trace

Note right of R: Trace chunk is full
R->>+G: PATCH trace [build.id, offset, data]
G->>+D: find or create chunk [chunk.index]
D-->>-G: chunk [id, index]
G->>I: append chunk data [chunk.index, data]
G->>G: chunk full [index]
G-->>-R: 200 OK
G->>+I: read chunk data [chunk.index]
I-->>-G: chunk data [data, size]
G->>O: send chunk data [data, size]
G->>+D: update data store type [chunk.id]
G->>+I: delete chunk data [chunk.index]
```

NFS coupling

In 2017, we experienced serious problems of scaling our NFS infrastructure. We even tried to replace NFS with
CephFS - unsuccessfully.

Since that time it has become apparent that the cost of operations and maintenance of a NFS cluster is significant and that if we ever decide to migrate to Kubernetes
we need to decouple GitLab from a shared local storage and NFS.

1. NFS might be a single point of failure
1. NFS can only be reliably scaled vertically
1. Moving to Kubernetes means increasing the number of mount points by an order of magnitude
1. NFS depends on extremely reliable network which can be difficult to provide in Kubernetes environment
1. Storing customer data on NFS involves additional security risks

Moving GitLab to Kubernetes without NFS decoupling would result in an explosion

of complexity, maintenance cost and enormous, negative impact on availability.

Iterations

1. · Implement the new architecture in way that it does not depend on shared local storage
1. · Evaluate performance and edge-cases, iterate to improve the new architecture
1. · Design cloud native build logs correctness verification mechanisms
1. · Build observability mechanisms around performance and correctness
1. · Rollout the feature into production environment incrementally

The work needed to make the new architecture production ready and enabled on GitLab.com had been tracked in Cloud Native Build Logs on GitLab.com epic.

Enabling this feature on GitLab.com is a subtask of
making the new architecture generally available for everyone.

Status

This change has been implemented and enabled on GitLab.com.

We are working on an epic to make this feature more resilient and observable.

SECTION: engineering/architecture/design-documents/cloud_native_gitlab_pages/_index.md

{{< engineering/design-document-header >}}

GitLab Pages is an important component of the GitLab product. It is mostly being used to serve static content, and has a limited set of well defined responsibilities. That being said, unfortunately it has become a blocker for GitLab.com Kubernetes migration.

Cloud Native and the adoption of Kubernetes has been recognised by GitLab to be one of the top two biggest tailwinds that are helping us grow faster as a company behind the project.

This effort is described in more detail
in the infrastructure team handbook page.

GitLab Pages is tightly coupled with NFS and to unblock Kubernetes migration a significant change to GitLab Pages' architecture is required. This is an ongoing work that we have started more than a year ago. This blueprint might be useful to understand why it is important, and what is the roadmap.

How GitLab Pages Works

GitLab Pages is a daemon designed to serve static content, written in
Go.

Initially, GitLab Pages has been designed to store static content on a local shared block storage (NFS) in a hierarchical group > project directory structure. Each directory, representing a project, was supposed to contain a configuration file and static content that GitLab Pages daemon was supposed to read and serve.

mermaid
graph LR
  A(GitLab Rails) -- Writes new pages deployment --> B[(NFS)]
  C(GitLab Pages) -. Reads static content .-> B

This initial design has become outdated because of a few reasons - NFS coupling being one of them - and we decided to replace it with more "decoupled service"-like architecture. The new architecture, that we are working on, is described in this blueprint.

NFS coupling

In 2017, we experienced serious problems of scaling our NFS infrastructure. We even tried to replace NFS with
CephFS - unsuccessfully.

Since that time it has become apparent that the cost of operations and maintenance of a NFS cluster is significant and that if we ever decide to migrate to Kubernetes
we need to decouple GitLab from a shared local storage and NFS.

1. NFS might be a single point of failure
1. NFS can only be reliably scaled vertically
1. Moving to Kubernetes means increasing the number of mount points by an order of magnitude
1. NFS depends on extremely reliable network which can be difficult to provide in Kubernetes environment
1. Storing customer data on NFS involves additional security risks

Moving GitLab to Kubernetes without NFS decoupling would result in an explosion of complexity, maintenance cost and enormous, negative impact on availability.

New GitLab Pages Architecture

- GitLab Pages sources domains' configuration from the GitLab internal API, instead of reading config.json files from a local shared storage.
- GitLab Pages serves static content from Object Storage.

mermaid
graph TD
  A(User) -- Pushes pages deployment --> B{GitLab}
  C((GitLab Pages)) -. Reads configuration from API .-> B
  C -. Reads static content .-> D[(Object Storage)]

C -- Serves static content --> E(Visitors)

This new architecture has been briefly described in
the blog post
too.

Iterations

1. · Redesign GitLab Pages configuration source to use the GitLab API
1. · Evaluate performance and build reliable caching mechanisms
1. · Incrementally rollout the new source on GitLab.com
1. · Make GitLab Pages API domains configuration source enabled by default
1. Enable experimentation with different servings through feature flags
1. Triangulate object store serving design through meaningful experiments
1. Design pages migration mechanisms that can work incrementally
1. Gradually migrate towards object storage serving on GitLab.com

GitLab Pages Architecture
epic with detailed roadmap is also available.

SECTION: engineering/architecture/design-documents/codebase_as_chat_context/_index.md

<!--

The canonical place for the latest set of instructions (and the likely source
of this file) is
content/handbook/engineering/architecture/design-documents/template.md.

Document statuses you can use:

- "proposed"
- "accepted"
- "ongoing"
- "implemented"
- "postponed"
- "rejected"

-->

<!-- Design Documents often contain forward-looking statements -->
<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

<!--
Don't add a h1 headline. It'll be added automatically from the title front matter attribute.

For long pages, consider creating a table of contents.
-->

Summary

We are introducing the capability to include Codebase as additional contexts to Duo Chat requests. This can refer to the entire repository or to a sub-directory under the repository.

To achieve this, we will index the codebase as vector embeddings, referred to as Code Embeddings.

When the user asks a question on Duo Chat, the system executes a semantic search over the Code Embeddings to retrieve relevant context from repositories, which is then processed by large language models to generate helpful responses.

This new feature will be available to GitLab Premium or Ultimate users with the Duo Pro or Duo Enterprise add-ons.

Epic: The work for this feature is tracked in this epic.

Motivation

Currently, we don't do a great job of helping customers understand their repository and code base. Competitors support a broader aperture -- a user can ask questions about an entire repository, or scope the context to multiple folders, multiple files, and portions of code.
This functional gap is commonly mentioned by customers, and here's a recent summary of research in this space. LLM's are only as good as the context we give them, so it is important that we achieve parity with our competitors in this area.

This initiative aims to bridge this critical functional gap in GitLab's Duo Chat offering by enabling users to interact with their entire codebase through natural language queries. This capability allows users to more effectively understand, navigate, and plan changes to their repositories -- a feature already offered by competing products.

Goals

The main goal is to add Codebase as an additional context to Duo Chat. In this initiative, this is scoped to Repository and Directory. A semantic search will then be done over the Repository or Directory, with the results used to enhance the Chat prompt sent to the AI model.

To support semantic search, the creation of code embeddings is included in the initial scope of this work. Indexing of the default branch will be done for the first phase of the work, with feature branches indexed in the second phase.

We will only generate embeddings for projects or namespaces with Duo enabled.

Non-Goals

The following is out of scope for this initiative, but could theoretically be built upon it:

- Codebase as additional context for Agentic Duo Chat.
- Codebase as additional context for Duo Chat Slash Commands.
- Codebase as additional context for Code Suggestions.
- Support for indexing and querying locally changed files as vector embeddings.
- A Knowledge Graph representation of the codebase as additional context to Duo Chat.

Please see Next Steps and Future Proofing for proposed plans regarding the above topics.

Proposal

In order to support Codebase as Chat Context, we need to:

1. Introduce Code Embeddings
    - This is a vector representation of files in the codebase.
    - This includes a pipeline to index the codebase as vector embeddings
    - This gives the ability to perform a semantic search over the embeddings
    - This will be developed in 2 phases:
        - Phase 1: Support code embeddings on the main branch
        - Phase 2: Support code embeddings on feature branches

1. Update Duo Chat to support codebase as additional context.
    - When asking a question on Chat, the user will have the option to include the following as contexts:
        - repository - refers to the entire codebase of a project
        - directory - this is a "subset" of the repository, and refers to a subfolder in a project
    - When a repository is selected as additional context
        - a semantic search is done over the Code Embeddings representation of the files in the repository. The search result is then used to enhance the Chat prompt sent to the AI model.
    - When a directory is selected as additional context:
        - a semantic search is done over the Code Embeddings representation of the files in the directory. The search result is then used to enhance the Chat prompt sent to the AI model.
        - as an additional consideration: if a directory only has a few files, the file contents are included directly as additional context to enhance the Chat prompt sent to the AI model.

Design and implementation details

Components

This initiative introduces or updates the following components:

Code Embeddings

Please refer to the Code Embeddings blueprint for the detailed design of this component.

This is a vector representation of files in the Codebase. We will introduce an indexing pipeline to generate embeddings when a change is pushed to a branch. We will also introduce the capability to search over these embeddings.

Indexing the Code Embeddings

- The changes are done on both the Gitlab Elasticsearch Indexer and GitLab Rails.
- On Rails, we will make use of the AI Context Abstraction Layer.
- We will make use of a new Code Parser library to parse the code files into logical chunks before generating embeddings on those chunks.
  - The Code Parser lives in its own repository so that it can be used in different projects.
  - For further design and implementation details, please see the One Parser proposal.

Searching the Code Embeddings

- The changes will be done on GitLab Rails
- We will make use of the AI Context Abstraction Layer to perform a search on the embeddings.

Duo Chat

Duo Chat is already an existing AI feature on GitLab.

For details on the current Duo Chat workflow and architecture, please refer to the following documentations:

- How a Chat prompt is constructed
- GraphQL API (aiAction) flow
- Duo Chat process flow
- Duo Chat tools

In this initiative, we will introduce changes on the GitLab Language Server, GitLab Rails, and the GitLab AI Gateway. For further implementation details, please proceed to the Adding the Codebase as Context on Duo Chat section below.

Adding the Codebase as Context on Duo Chat

The diagram below shows the workflow for including codebase (repository or directory) semantic search results as additional context. The areas highlighted in blue are where we'll introduce new changes.

```
mermaid
sequenceDiagram
    actor USR as User
    participant FE as IDE/Language Server
    box GitLab Rails
      participant GLRGQL as GraphQL API
      participant GLRLLMS as LLM Services
      participant GLRLLMCHAT as LLM Chat Module
      participant GLRLLMSEM as LLM Semantic Search Tool
      participant CES as Code Embeddings Search Service
    end
    participant CE as Code Embeddings Storage
    participant AIGW as AI Gateway
    participant LLM as LLM
```

USR->>FE: Asks a question, indicating
 repository or directory
 as additional context

FE->>GLRGQL: Sends chat request to aiAction mutation
 with repository or directory
 as additional context
Note over FE: The Language Server will send the ID and category
 of the repository and directory additional contexts,
 but the content will still be empty at this point.
GLRGQL->>GLRLLMS: Sends chat request, with
 repository or directory
 as additional context
GLRLLMS->>GLRLLMCHAT: Sends chat request, with
 repository or directory
 as additional context

rect rgb(240, 248, 255)
GLRLLMCHAT->>GLRLLMSEM: Executes the Semantic Search Tool,
 with a repository or directory filter
GLRLLMSEM->>CES: Performs the semantic search

CES->>AIGW: Generates embeddings for the question
AIGW-->>CES: Returns embeddings for the question
CES->>CE: Queries Code Embeddings with
 the question embeddings as target
 and filtered by the given repository or directory
CE-->>CES: Returns Code Embeddings
CES-->>GLRLLMSEM: Returns semantic search result

GLRLLMSEM-->>GLRLLMCHAT: Returns semantic search result
GLRLLMCHAT->>GLRLLMCHAT: Adds the semantic search result as the content
 to the repository and directory additional context
Note over GLRLLMCHAT: The content of the repository and directory
 additional contexts will be set here.

GLRLLMCHAT->>AIGW: Request to /v2/chat/agent
 with repository or directory +
 semantic search result as additional context
AIGW->>AIGW: Builds a prompt with the
 repository or directory +
 semantic search result
 as additional context
end

AIGW->>LLM: Sends prompt with the additional contexts

LLM-->>AIGW: Returns the final answer
AIGW-->>GLRLLMSEM: Returns the final answer
GLRLLMSEM-->>GLRLLMCHAT: Returns the final answer
GLRLLMCHAT-->>GLRLLMS: Returns the final answer
GLRLLMS-->>GLRGQL: Returns the final answer
GLRGQL-->>FE: Returns the final answer
FE-->>USR: Shows the answer

Additional Context Category

We will add one additional context category: repository. If a directory is given, it will be

considered a repository additional context, with the relative path of the directory specified in the metadata.

Unit Primitives

We will add the following unit primitives as part of this initiative:

Include Context

- includerepositorycontext - used by both the Language Server and AIGW

Tool

- codebasesearch - used by Rails, for the Semantic Search Tool

Embeddings Generation

- generateembeddingscodebase - unit primitive used for the embeddings generation endpoint (/v1/proxy/vertex-ai)

Code Embeddings Search Service

This is a service class that handles the calls to the Code Embeddings. This makes use of the AI Context Abstraction Layer. For details on how this is done, please refer to the Search section in the Code Embeddings blueprint.

Duo Semantic Search Tool

This is a new Duo Chat tool that will be introduced in this initiative.

Similar to the Slash Command Tools, there is no need to have LLM infer whether the tool is needed. This tool will be called as long as the repository additional context is present.

On Rails, we will introduce an LLM Tool class that will call the Code Embeddings Search Service to fetch the matching embeddings of a Chat question. This new Semantic Search Tool will be called from the LLM Chat module, with the search results then included as the content of either the repository additional context. The Chat request is then sent to AIGW with the new additional contexts.

API Changes - Duo Chat Available Features

The Language Server calls the GraphQL query {currentUser { duoChatAvailableFeatures } } to fetch the list of available Duo Chat features.

As part of this initiative, we will add the includerepositorycontext unit primitive to this list of features.

API Changes - Chat Request

The GraphQL mutation used by Duo Chat (aiAction) already accepts additionalContext as a

parameter for a chat input.

With the repository additional context category, the chat input to the aiAction mutation should then look like:

For repository as additional context

```
graphql
mutation newChatMessage {
  aiAction(
    input: {
      chat: {
        content: "the user question"
        additionalContext: [{
          category: "repository",
          id: "the-project-id",
          content: "",  should be empty
          metadata: {
            directory: "some/dir"  this is optional, specified when the user selects directory
as an additional context
            branch: "some-branch"  including the branch is a second phase iteration
          }
        }]
      }
    }
  ) {
    requestId
  }
}
```

Duo Chat Changes - Frontend

See UI Design.

Evaluations

Codebase context enhancement can have different results depending on different factors such as the granularity of embeddings or the embeddings model used. Beyond the MVC iteration of this feature, we should evaluate the effectivity of different embeddings models, chunking granularity, and other approaches to embeddings.

Possible approaches for evaluation

Approach	Description
Size-based chunking	Split files into chunks of fixed size or token count
Tree-sitter chunking	Parse code structure using AST to create semantically meaningful chunks
Whole File Embedding	Generate embeddings for entire file contents (blob content)

| Different Embedding Models | Use purpose-built models for code vs. general text models |

Next Steps and Future Proofing

Proposed steps for porting to the Agentic Chat architecture

Once we introduce the Agentic Chat architecture, either the Duo Workflow Service on the AI Gateway or the Duo Workflow Executor on the Language Server will need to query the vector embeddings.

In order to support this, we will introduce an API over the Code Embeddings Search Service to be called either from the Duo Workflow Service or the Duo Workflow Executor.

Codebase as additional context for Duo Chat Slash Commands

Slash commands include /refactor, /fix, /test.

The Slash commands can either make use of the Semantic Search Tool or directly call the Code Embeddings Search Service to search over Code Embeddings.

Alternatively, we can support codebase as additional context for Slash commands only in Agentic Chat.

Codebase as additional context for Code Suggestions

Code Completion or Code Generation can make use of the Code Embeddings Search Service, which abstracts all the logic needed for searching over the Code Embeddings.

Proposed steps for supporting local file indexing

TBA

Indexing the codebase as a Knowledge Graph

TBA

Alternative Solutions

Allow LLM to infer the need for Codebase Semantic Search

In this solution, we would introduce a tool definition for the Codebase Semantic Search on the AIGW. This tool will then be made available to the LLM, which infers whether the tool is needed based on the question.

We decided not to go with this solution for the following reasons:

- There is no need for the LLM to infer whether the codebase search tool is required. If a repository or directory additional context is included, then we can immediately do a codebase search.
- The MVC proposal and the requirement from Product is that the user should be able to

explicitly decided whether a codebase semantic search is needed. In this solution, while the user can specify repository or directory as additional context, the LLM still has the final decision on whether the codebase search is performed.
- This solution means an additional round of requests between Rails and AIGW, making the latency higher.

PoC for the tool definition: MR: POC: Codebase search tool.

Workflow diagram:

```
mermaid
sequenceDiagram
    actor USR as User
    participant FE as IDE/Language Server
    box GitLab Rails
      participant GLRGQL as GraphQL API
      participant GLRLLMS as LLM Services
      participant GLRLLMCHAT as LLM Chat Module
      participant GLRLLMSEM as LLM Semantic Search Tool
      participant CES as Code Embeddings Search Service
    end
    participant CE as Code Embeddings Storage
    participant AIGW as AI Gateway
    participant LLM as LLM

    USR->>FE: Asks a question, indicating<br /> codebase as additional context

    FE->>GLRGQL: Sends chat request to aiAction mutation<br /> with codebase as additional
context
    GLRGQL->>GLRLLMS: Sends chat request, with<br /> codebase as additional context
    GLRLLMS->>GLRLLMCHAT: Sends chat request, with<br /> codebase as additional context
    GLRLLMCHAT->>AIGW: Request to /v2/chat/agent<br /> with codebase as additional context

    rect rgb(240, 248, 255)
      AIGW->>LLM: Sends chat request, indicating<br /> that codebase context is present
      LLM->>LLM: Determines that the<br /> semantic search tool is needed
      Note over LLM: The LLM will have a new available<br /> tool for semantic search.<br />
This tool will have instructions<br /> to perform a semantic search if<br /> the codebase
context is present.
      LLM-->>AIGW: Returns response, indicating that<br /> the semantic search tool is needed
    end

    AIGW-->>GLRLLMCHAT: Returns response, indicating that<br /> the semantic search tool is
needed

    GLRLLMCHAT->>GLRLLMSEM: Executes the Semantic Search Tool

    rect rgb(240, 248, 255)
      GLRLLMSEM->>CES: Performs the semantic search
      CES->>AIGW: Generates embeddings for the question
```

```
    AIGW-->>CES: Returns embeddings for the question
    CES->>CE: Queries Code Embeddings with<br /> the question embeddings as target
    CE-->>CES: Returns Code Embeddings
    CES-->>GLRLLMSEM: Returns semantic search result
  end

  GLRLLMSEM->>AIGW: Request to /v1/prompts/chat<br />with search results as additional
context

  AIGW->>AIGW: Builds a prompt with the<br /> search results as additional context
  AIGW->>LLM: Sends prompt with the<br /> search results as additional context

  LLM-->>AIGW: Returns the final answer
  AIGW-->>GLRLLMSEM: Returns the final answer
  GLRLLMSEM-->>GLRLLMCHAT: Returns the final answer
  GLRLLMCHAT-->>GLRLLMS: Returns the final answer
  GLRLLMS-->>GLRGQL: Returns the final answer
  GLRGQL-->>FE: Returns the final answer
  FE-->>USR: Shows the answer
```

Introduce an "Ask Codebase" tool and prompt

In this solution, we will introduce an "Ask Codebase" tool:

- On Rails, the LLM Chat module will determine that the tool is needed if there is a repository additional context
- The "Ask Codebase" tool will use the "Codebase Embeddings Search Service" to get the semantic search results
- The "Ask Codebase" tool will then send a request to the /v1/prompts/chat endpoint
- On AIGW, there will be a new prompt for askcodebase available through the /v1/prompts/chat endpoint
- The workflow will essentially be similar to the Slash Command tools workflow

```
mermaid
sequenceDiagram
  actor USR as User
  participant FE as IDE/Language Server
  box GitLab Rails
    participant GLRGQL as GraphQL API
    participant GLRLLMS as LLM Services
    participant GLRLLMCHAT as LLM Chat Module
    participant GLRLLMAC as LLM Ask Codebase Tool
    participant CES as Code Embeddings Search Service
  end
  participant CE as Code Embeddings Storage
  participant AIGW as AI Gateway
  participant LLM as LLM

  USR->>FE: Asks a question, indicating<br /> repository or directory<br /> as additional
```

context

FE->>GLRGQL: Sends chat request to aiAction mutation
 with repository or directory
 as additional context
Note over FE: The Language Server will send the ID and category
 of the repository and directory additional contexts,
 but the content will still be empty at this point.
GLRGQL->>GLRLLMS: Sends chat request, with
 repository or directory
 as additional context
GLRLLMS->>GLRLLMCHAT: Sends chat request, with
 repository or directory
 as additional context

rect rgb(240, 248, 255)
GLRLLMCHAT->>GLRLLMAC: Executes the Ask Codebase Tool,
 with a repository or directory filter
GLRLLMAC->>CES: Performs the semantic search

CES->>AIGW: Generates embeddings for the question
AIGW-->>CES: Returns embeddings for the question
CES->>CE: Queries Code Embeddings with
 the question embeddings as target
 and filtered by the given repository or directory
CE-->>CES: Returns Code Embeddings
CES-->>GLRLLMAC: Returns semantic search result

GLRLLMAC-->>GLRLLMCHAT: Returns semantic search result
GLRLLMCHAT->>GLRLLMCHAT: Adds the semantic search result as additional context

GLRLLMCHAT->>AIGW: Request to /v1/prompts/chat
 with repository or directory +
 semantic search result as additional context
AIGW->>AIGW: Builds a prompt with the
 repository or directory +
 semantic search result
 as additional context
Note over AIGW: This will be a new askcodebase prompt
end

AIGW->>LLM: Sends prompt with the additional contexts

LLM-->>AIGW: Returns the final answer
AIGW-->>GLRLLMAC: Returns the final answer
GLRLLMAC-->>GLRLLMCHAT: Returns the final answer
GLRLLMCHAT-->>GLRLLMS: Returns the final answer
GLRLLMS-->>GLRGQL: Returns the final answer
GLRGQL-->>FE: Returns the final answer
FE-->>USR: Shows the answer

Using different categories and unit primitives for repository and directory

Instead of using a single repository category for the repository and directory additional context, we will use repository and directory categories.

Note: we decided not to go with this option because a category has a 1-to-1 mapping to a unit

primitive, and conceptually, we should only have 1 unit primitive for the "include codebase" context.

The chat input to the aiAction mutation should then look like:

For repository as additional context

```
graphql
mutation newChatMessage {
  aiAction(
    input: {
      chat: {
        content: "the user question"
        additionalContext: [{
          category: "repository",
          id: "the-project-id",
          content: "", // should be empty
          metadata: {'branch': 'some-branch'} // including the branch is a second phase
iteration
        }]
      }
    }
  ) {
    requestId
  }
}
```

For directory as additional context

```
graphql
mutation newChatMessage {
  aiAction(
    input: {
      chat: {
        content: "the user question"
        additionalContext: [{
          category: "directory",
          id: "file:///home/user/workspace/src/dir",
          content: "", // should be empty
          metadata: {
            'relativePath': 'src/dir',
            'branch': 'some-branch' // including the branch is a second phase iteration
          }
        }]
      }
    }
  ) {
    requestId
  }
```

}

SECTION: engineering/architecture/design-documents/codebase_as_chat_context/code_embeddings.md

Code Embeddings

This blueprint is being written up in the merge request Blueprint: Codebase as Chat Context: Code Embeddings

---
SECTION: engineering/architecture/design-documents/code_search_with_zoekt/_index.md

{{< engineering/design-document-header >}}

Summary

We have implemented an additional code search functionality in GitLab that
is backed by Zoekt, an open source
search engine specifically designed for code search. Zoekt is used as
an API by GitLab and remains an implementation detail, while the user interface
in GitLab has been enhanced with new features enabled by Zoekt's capabilities.

This integration provides significant improvements over the existing Elasticsearch-based search, including:

1. Exact match mode: Returns results that precisely match the search query, eliminating false positives
1. Regular expression mode: Supports regex patterns and boolean expressions for powerful code searching
1. Multiple line matches: Shows multiple matching lines from the same file in the search results
1. Self-registering architecture: Enables simple scaling and management of search infrastructure

Motivation

GitLab code search functionality has historically been backed by Elasticsearch.
While Elasticsearch has proven useful for other types of search (issues, merge
requests, comments, etc.), it is not ideally suited for code search where users expect matches to be precise (no false positives) and
flexible (supporting features like substring matching
and regexes).

After investigating our options,
we determined that Zoekt is the most suitable
well-maintained open source technology for code search. Our research indicated that

the fundamental architecture of Zoekt matches what we would implement if we were to build a solution from scratch.

Our benchmarking
showed that Zoekt is viable at our scale, and the integration has been successfully deployed to GitLab.com.

Goals

The main goals of this integration have been to implement the following highly requested improvements to code search:

1. Exact match (substring match) code searches in advanced search
1. Support regular expressions with Advanced Global Search
1. Support multiple line matches in the same file

The rollout was designed to catch and resolve scaling or infrastructure cost issues as early as possible, allowing us to pivot if necessary before investing too heavily in this technology.

Non-Goals

The following were not initial goals but could be built upon this solution in the future:

1. Improving security scanning features by leveraging fast regex scans across repositories
1. Reducing search infrastructure costs (though this may be possible with further optimizations)
1. AI/ML features to predict what users might be interested in finding
1. Comprehensive code intelligence and navigation features (which would require more structured data)

Proposal

An initial implementation of the Zoekt integration
was created to demonstrate the feasibility of using Zoekt as a drop-in
replacement for Elasticsearch code searches. This design document outlines the details of the implementation and the steps taken to scale the solution for GitLab.com and self-managed instances.

Design and implementation details

User Experience

When a user performs an advanced search on a group or project where Zoekt is enabled, they can now toggle between two search modes in the UI:

- Exact match mode: Returns results that exactly match the query (default mode)
- Regular expression mode: Supports regex patterns and boolean expressions

Users can select their preferred search mode using a toggle in the UI. The search syntax

supports advanced filtering with modifiers like:

- file: to filter by filename
- lang: to filter by programming language
- sym: to search within symbols (methods, classes, etc.)
- and other syntax options

Here's a screenshot of the new UI:

Key Components

Unified Binary: gitlab-zoekt

Zoekt comes with it's own binaries for indexing and searching. Initially we used some of these and we started to build out our own binaries over time. We then pivoted to a single binary for both indexing and searching.
We call this unified binary gitlab-zoekt, which replaces the previously separate binaries (gitlab-zoekt-indexer and gitlab-zoekt-webserver). This is a Go codebase which uses public modules from the Zoekt codebase as a library, rather than using the binaries directly. This unified binary can operate in two distinct modes:

- Indexer mode: Responsible for indexing repositories
- Webserver mode: Responsible for serving search requests

Having a unified binary simplifies deployment, operation, and maintenance of the Zoekt infrastructure. The key advantages of this approach include:

1. Simplified deployment: Only one binary needs to be built, deployed, and maintained
1. Consistent codebase: Shared code between indexer and webserver is maintained in one place
1. Operational flexibility: The same binary can run in different modes based on configuration
1. Testing mode: The unified binary can run both services simultaneously for testing purposes

Database Models

Zoekt is not a distributed database (like Elasticsearch) or even really a database service (like Postgres) but instead it's a set of Go modules (and binaries) that interact with index files on disk. It supports creating index files and searching them. Since we needed to build a higher level distributed, clustered and replicated search engine on top of it we needed to manage all of the lifecycle of Zoekt processes and indexes somewhere. We chose to store all this lifecycle data in Rails and Zoekt processes periodically poll Rails state to figure out what to do next.

GitLab uses several database models to manage Zoekt:

- Search::Zoekt::EnabledNamespace: Tracks which top-level namespaces have Zoekt enabled
- Search::Zoekt::Node: Represents a Zoekt server node with information about its capacity, status, and configuration
- Search::Zoekt::Replica: Manages replica relationships for high availability

- Search::Zoekt::Index: Manages the index state for a top level namespace, including storage allocation and watermark levels
- Search::Zoekt::Repository: Represents a project repository in Zoekt with indexing state
- Search::Zoekt::Task: Tracks indexing tasks (index, forceindex, delete) that need to be processed by Zoekt nodes

Architecture Overview

```mermaid
graph TD
    User[User] --> GitLab[GitLab Rails Application]
    GitLab <--> DB[(GitLab Database)]
    GitLab <--> ZoektNode1[Zoekt Node 1]
    GitLab <--> ZoektNode2[Zoekt Node 2]
    GitLab <--> ZoektNodeN[Zoekt Node N]
    ZoektNode1 <--> Gitaly[Gitaly]
    ZoektNode2 <--> Gitaly
    ZoektNodeN <--> Gitaly

    subgraph "Zoekt Node"
        ZoektBinary["gitlab-zoekt binary"]
        IndexStorage[(Index Storage)]
        Gateway[NGINX Gateway]
        ZoektBinary --> IndexStorage
        Gateway --> ZoektBinary
    end

    subgraph "Database Models"
        Node[Search::Zoekt::Node]
        Index[Search::Zoekt::Index]
        Task[Search::Zoekt::Task]
        Repository[Search::Zoekt::Repository]
        EnabledNamespace[Search::Zoekt::EnabledNamespace]
        Replica[Search::Zoekt::Replica]
    end
```

The Zoekt integration consists of several key components working together:

1. GitLab Rails Application: Manages which repositories need to be indexed, coordinates with Zoekt nodes
1. Zoekt Nodes: Run the gitlab-zoekt binary to handle indexing and searching of repositories
1. Gitaly: Provides Git repository access to Zoekt for indexing
1. Database: Stores metadata about nodes, indices, tasks, and repositories

Indexing Flow

```mermaid
sequenceDiagram
    participant GitLab as GitLab Rails
```

```
    participant DB as GitLab Database
    participant Zoekt as Zoekt Node
    participant Gitaly as Gitaly Service

    Note over GitLab: Repository updated or created
    GitLab->>DB: Create zoekttasks records

    loop Task Polling
        Zoekt->>GitLab: GET /internal/search/zoekt/:uuid/tasks
        GitLab->>DB: Find pending tasks
        GitLab->>Zoekt: Return tasks to process
    end

    Zoekt->>Gitaly: Fetch repository data
    Zoekt->>Zoekt: Create/update search index
    Zoekt->>GitLab: POST /internal/search/zoekt/:uuid/callback
    GitLab->>DB: Update task status
    GitLab->>DB: Update repository and index state
```

The indexing process follows these steps:

1. When a repository is created or updated, the GitLab Rails application creates zoekttasks records
1. Zoekt nodes (running in indexer mode) periodically pull tasks through the internal API
1. Zoekt nodes process the tasks by fetching repository data from Gitaly and creating search indices
1. Zoekt nodes send callback notifications to GitLab to update task status
1. GitLab updates the appropriate database records (zoekttask, zoektrepository, zoektindex)

zoekttask can be of three different types:

- indexrepo: Incremental indexing (from the last indexed SHA to the latest SHA of the default branch)
- forceindexrepo or forced indexing: Full reindex of the repository (deletes existing index files and reindexes everything)
- deleterepo: Schedules existing indexed files for deletion

To avoid race conditions, there is a locking mechanism to ensure only one indexing operation occurs for a project at any given time.

Search Flow

```
mermaid
sequenceDiagram
    participant User
    participant GitLab as GitLab Rails
    participant DB as GitLab Database
    participant Zoekt as Zoekt Node (Webserver)
```

```
User->>GitLab: Perform code search
GitLab->>DB: Check if namespace has Zoekt enabled
GitLab->>DB: Get online Zoekt nodes
Note over GitLab: Apply user permissions
GitLab->>Zoekt: Forward search query to node
Zoekt->>Zoekt: Process search query against index
Zoekt->>GitLab: Return search results
GitLab->>User: Format and present results
```

The search process follows these steps:

1. User performs a search in GitLab UI
1. GitLab determines if the search should use Zoekt based on user preferences and enabled namespaces
1. If Zoekt is appropriate, GitLab forwards the search to a Zoekt node running in webserver mode
1. Zoekt processes the search and returns results
1. GitLab formats and presents the results to the user

Communication Flow

The communication between GitLab and Zoekt nodes happens through bidirectional API calls, all secured with appropriate authentication mechanisms.

Authentication Architecture

The Zoekt integration implements a comprehensive authentication system with these key components:

1. Indexer · Rails Authentication (JWT): Zoekt indexer authenticates to GitLab Rails using JWT tokens signed with the GitLab shell secret
2. Rails · Webserver Authentication (Basic Auth): GitLab Rails authenticates to Zoekt webserver using HTTP Basic Authentication via NGINX
3. Future Planned Authentication (JWT): Plans to replace Basic Auth with JWT for Rails · Webserver authentication, mirroring the approach used for Indexer · Rails

This tiered authentication approach ensures secure communication in all directions while maintaining compatibility with GitLab's existing security patterns.

Task Retrieval API

Zoekt nodes periodically call GitLab's internal API to:

- Register themselves with GitLab (providing node information like UUID, URL, disk space)
- Retrieve tasks that need to be processed
- Update their status and metrics

http
GET /internal/search/zoekt/:uuid/tasks

This API is secured with JWT authentication, where:

- The JWT token is generated by the Zoekt indexer using the GitLab shell secret
- The token is included in the Gitlab-Shell-Api-Request header
- GitLab Rails validates the token using the same shell secret

This authentication mechanism ensures that only authorized Zoekt nodes can register and retrieve tasks from GitLab.

Callback API

After processing tasks, Zoekt nodes call GitLab's callback API to:

- Update task status (success/failure)
- Provide additional information (for example, repository size)
- Report errors or issues

http
POST /internal/search/zoekt/:uuid/callback

This API also uses JWT authentication with the same mechanism as the Task Retrieval API, ensuring secure bidirectional communication.

This asynchronous callback architecture is a significant improvement over the previous design, which used Sidekiq jobs for indexing operations. By using callbacks instead of blocking Sidekiq jobs, the system gains several important benefits:

1. Reduced Sidekiq load: Indexing operations no longer block Sidekiq workers, freeing them for other critical GitLab tasks
1. Better scalability: The number of concurrent indexing operations is only limited by Zoekt node capacity, not by Sidekiq worker availability
1. Improved reliability: If a node goes down during indexing, it doesn't leave Sidekiq jobs in an incomplete state
1. More efficient resource usage: Long-running indexing tasks don't consume valuable Sidekiq resources
1. Separation of concerns: Zoekt nodes handle indexing independently, reporting back only when completed

This approach allows GitLab to maintain a lightweight coordination role while the computationally intensive work is handled by specialized Zoekt nodes, resulting in better overall system performance and responsiveness.

Search API

GitLab calls the Zoekt webserver API to:

- Execute search queries
- Retrieve search results

- Apply filtering based on user permissions

http
GET /api/search

In deployed environments (particularly with Helm), this communication is secured with HTTP Basic Authentication configured in NGINX. This provides a simple but effective authentication layer for search requests.

The authentication approach for search is planned to transition to JWT-based authentication in the future, which will provide more granular control and better align with GitLab's authentication patterns.

Zoekt Infrastructure

Each Zoekt node runs a single gitlab-zoekt binary that can operate in both indexer and webserver modes simultaneously. The nodes store .zoekt index files on persistent storage for fast searches.

A typical deployment includes:

- The gitlab-zoekt binary serving both indexing and search requests
- Universal CTags for symbol extraction
- An internal NGINX gateway for routing requests

Scaling and High Availability

Self-Registering Node Architecture

Zoekt implements a self-registering node architecture inspired by GitLab Runner:

1. Zoekt nodes register themselves with GitLab by providing their address, name, and status
2. GitLab maintains a registry of nodes with their status, capacity, and assignments
3. GitLab manages the shard assignments internally, assigning namespaces to specific nodes
4. Nodes that don't check in for a configurable period can be automatically removed

This architecture makes the system self-configuring and facilitates easy scaling.

Unlike the GitLab Runner the Zoekt nodes authenticate with a shared secret managed at the infrastructure level and cannot be registered by users. So self-registration is more of a convenience for the operator rather than a feature for users.

Sharding Strategy

1. Groups/namespaces are assigned to specific Zoekt nodes for indexing and searching
1. GitLab manages the shard assignments internally based on node capacity and load
1. When new nodes are added, they can automatically take on new workloads
1. If nodes go offline, their work can be reassigned to other nodes

Replication Strategy

```
mermaid
graph TD
    GitLab[GitLab Rails Application]
    DB[(Database)]

    GitLab <--> DB

    subgraph "Database Models"
        ReplicaRecord1["Replica Record 1"]
        ReplicaRecord2["Replica Record 2"]
        Index1["Index 1 for Namespace A\n(on Node 1)"]
        Index2["Index 2 for Namespace A\n(on Node 2)"]
    end

    ReplicaRecord1 --> Index1
    ReplicaRecord2 --> Index2

    subgraph "Physical Infrastructure"
        Node1[Zoekt Node 1]
        Node2[Zoekt Node 2]
        Gitaly[Gitaly]

        Node1 <--> Gitaly
        Node2 <--> Gitaly
    end

    Index1 -.-> Node1
    Index2 -.-> Node2

    GitLab --> Node1
    GitLab --> Node2
```

The replication strategy works at the database record level rather than through actual data synchronization between nodes:

1. Independent Indexing: Each Zoekt node independently indexes repositories by fetching data directly from Gitaly
1. Multiple Replica Records: For high availability, GitLab can create multiple Search::Zoekt::Replica records for a single namespace
1. Distributed Indices: Each replica record is associated with an index record that may be assigned to different physical Zoekt nodes
1. Fast Indexing: Indexing is efficient (approximately 10 seconds for a large repository like gitlab-org/gitlab), making it practical to maintain multiple independent indices
1. No Complex Synchronization: This approach eliminates the need for complex index file synchronization between nodes
1. Search Load Distribution: GitLab can route search requests to any node that has an index for the relevant namespace

Currently, GitLab typically creates a single replica record per namespace, but the system is designed to support a configurable number of replicas per namespace in the future. This approach provides the following benefits:

- Horizontal Scalability: Add more nodes to handle more namespaces or increase replication
- High Availability: If one node fails, searches can be routed to other nodes with replica indices
- Simple Operation: No complex replication mechanisms to maintain or troubleshoot
- Independent Scaling: Search and indexing capacity can be scaled independently by adding more nodes

This design prioritizes operational simplicity and reliability while still providing the necessary redundancy for high availability.

Since our Zoekt database is not a source of truth (ie. it simply syncing repos from Gitaly) we do not need to worry about assigning specific replicas to be a "primary" or "leader". Instead we just let the replicas independently sync data from Gitaly and assume that each time they update the index they will get the new source of truth.

Deployment Options

Kubernetes/Helm

GitLab provides a Helm chart (gitlab-zoekt) for Kubernetes deployments with the following features:

- Deploys Zoekt in a StatefulSet with persistent volumes for index storage
- Configurable resource allocation, scaling, and networking options
- Automatic node registration and service discovery
- Gateway component for load balancing and authentication
- Configurable Basic Authentication through NGINX for Rails · Webserver communication

The gitlab-zoekt Helm chart has proven to be highly scalable in production environments. On GitLab.com, this deployment is handling over 36 TiB of data, demonstrating its ability to operate at enterprise scale. The chart's design allows for both horizontal and vertical scaling to accommodate growing code search needs while maintaining performance and reliability.

Docker/Container

Containers are built from the CNG repository with:

- The unified gitlab-zoekt binary
- Universal CTags for symbol extraction
- Configurable environment variables for different operating modes

Database Schema

Key database tables include:

- zoektnodes: Information about Zoekt server nodes

- zoektindices: Tracks the indexing state for namespaces
- zoektrepositories: Maps GitLab projects to Zoekt indices
- zoekttasks: Queue of indexing tasks to be processed
- zoektenablednamespaces: Configuration for which namespaces use Zoekt
- zoektreplicas: Manages replica relationships for high availability

Database Model Relationships

The following diagram illustrates the relationships between the database models:

```
mermaid
classDiagram
    class Node
    class Index
    class Repository
    class Task
    class EnabledNamespace
    class Replica

    Node "1" --> "" Task : hasmany tasks
    Node "1" --> "" Index : hasmany indices

    EnabledNamespace "1" --> "" Replica : hasmany replicas

    Replica "1" --> "" Index : hasmany indices

    Index "1" --> "" Repository : hasmany repositories
    Repository "1" --> "" Task : hasmany tasks
```

Here's an example of the database structure for a namespace with multiple replicas and indices:

```
mermaid
graph TD
    Namespace["Namespace (gitlab-org)"]

    Replica1["Replica 1"]
    Replica2["Replica 2"]

    Index1A["Index 1A (Node 1)"]
    Index1B["Index 1B (Node 2)"]
    Index2A["Index 2A (Node 3)"]
    Index2B["Index 2B (Node 4)"]

    Repo1["Repository: gitlab"]
    Repo2["Repository: omnibus-gitlab"]
    Repo3["Repository: gitaly"]
    Repo4["Repository: gitlab-runner"]

    Namespace --> Replica1
```

```
Namespace --> Replica2

Replica1 --> Index1A
Replica1 --> Index1B
Replica2 --> Index2A
Replica2 --> Index2B

Index1A --> Repo1
Index1A --> Repo2
Index1B --> Repo3
Index1B --> Repo4

Index2A --> Repo1
Index2A --> Repo2
Index2B --> Repo3
Index2B --> Repo4
```

In this example:

- A namespace (gitlab-org) has Zoekt enabled through an EnabledNamespace record
- Two replica records are created for this namespace
- Each replica has an associated index record, assigned to different physical nodes
- Each index contains repositories for multiple projects within the namespace
- Tasks are created for each repository, tracking their indexing state on the respective nodes

This structure enables high availability and load distribution while maintaining a clear organization of the relationship between namespaces, indices, nodes, and repositories.

Current Development

Federated Search Using gRPC

A new gRPC-based federated search capability is being developed to enhance search performance across multiple Zoekt nodes. This feature replaces the previous HTTP-based search proxying by using a more efficient gRPC streaming implementation.

JWT Authentication for Rails · Webserver

In addition to the gRPC improvements, there are plans to implement JWT-based authentication for the Rails · Webserver communication flow. This would replace the current Basic Authentication approach with a more secure JWT implementation that mirrors the existing Indexer · Rails authentication. The goal is to provide a consistent, unified authentication strategy across all communication channels while leveraging GitLab's existing security infrastructure.

The gRPC federated search offers several advantages:

1. More efficient communication: gRPC uses HTTP/2 for transport, providing better performance than HTTP/1.1
1. Streaming between Zoekt nodes: Results are streamed between Zoekt nodes as they're found,

allowing the coordinating node to stop requesting results from other nodes once it has gathered enough matches
1. Reduced latency: Faster response times, especially for searches across many repositories, achieved through:
   - Concurrent searches across multiple nodes
   - Early termination once enough results are collected
   - More efficient binary protocol with HTTP/2
   - Processing results as they arrive rather than waiting for complete result sets
1. Better resource management: More granular control over search processing limits

It's important to note that while this implementation streams results between Zoekt nodes, the final results are still collected by the coordinating Zoekt node before being returned to Rails. The current implementation does not stream results to Rails. Instead, the performance benefits come from more efficient inter-node communication and the ability to stop searching once sufficient results are found, rather than exhaustively searching all repositories.

Configuration Options

The GitLab Zoekt integration can be configured through:

1. GitLab Admin Settings: Enable/disable indexing and searching, configure concurrent indexing tasks, set auto-deletion settings
1. User Preferences: Enable/disable exact code search for individual users
1. Zoekt Node Settings: Resource allocation, storage configuration, network settings
1. Feature Flags: Control specific features or behaviors of the integration

Rollout Strategy

The rollout strategy has followed these steps:

- [x] Initial availability for gitlab-org group
- [x] Improvements to monitoring and performance
- [x] Expansion to select customers with high code search needs
- [x] Implementation of sharding and replication for scalability
- [x] Gradual rollout to more licensed groups
- [x] Implementation of automatic balancing of shards
- [x] Assessment of costs and performance for broader rollout
- [x] Continued performance improvements
- [x] Availability to the majority of licensed groups on GitLab.com
- [ ] General availability to all licensed groups on GitLab.com (pending)

For self-managed instances, administrators can enable Zoekt by installing the required components and enabling the feature in the admin area.

Monitoring and Maintenance

To monitor the health and performance of the Zoekt integration, GitLab provides:

1. Admin UI: Shows indexing status, node health, and storage utilization
1. Rake Tasks: Tools to check indexing status. For example,

gitlab:zoekt:info

1. Automated Management: Features to automatically delete offline nodes, manage watermark levels, and redistribute indices
1. Logging: Detailed logging of indexing operations, search queries, and errors
1. Metrics: Performance metrics for indexing and search operations

Watermark Management

The Zoekt integration implements a sophisticated watermark management system that operates at both the node level and index level to ensure efficient storage utilization while preventing resource exhaustion.

Node-Level Watermarks

Each Zoekt node has watermark thresholds based on the percentage of total disk space used:

1. Low Watermark (60%): When disk usage exceeds this threshold, GitLab starts taking proactive measures to avoid reaching higher levels
1. High Watermark (75%): Signals potential storage pressure and prioritizes rebalancing actions
1. Critical Watermark (85%): May pause new indexing operations to prevent node overload while performing evictions

These node-level watermarks are used for overall node health monitoring and to make decisions about task assignment and index reallocation.

Index-Level Watermarks

In addition to node-level watermarks, each index within a node has its own watermark levels based on the ratio of used storage to reserved storage:

1. Ideal Storage Utilization (60%): Target level for optimal operation
1. Low Watermark (70%): Triggers evaluation for potential rebalancing or increasing reserved storage allocation
1. High Watermark (75%): Indicates the index is consuming more storage than expected and may prompt additional storage allocation if available
1. Critical Watermark (80%): May trigger eviction processes for this specific index or, if storage is available, a significant increase in reserved storage allocation

Each index has an associated watermarklevel enum state that reflects its current status:

- healthy: Operating within expec